

Préparation au Concours National Commun — Informatique MP

Épreuve d'Informatique

Filière MP — Durée : 4 heures

*Finalité dans les chaînes de blocs : le protocole Casper*d'après V. Buterin et V. Griffith, *Casper the Friendly Finality Gadget*,

Ethereum Foundation, arXiv:1710.09437v4 (2019)

Auteur : **Pr Agrégé El Hadiq Zouhair**cpgeacademy.org — Tous droits réservés © cpgeacademy.org

Avis important. Ce document est une **initiative personnelle** du professeur, à but strictement pédagogique. Il ne s'agit **pas** d'un sujet officiel du Concours National Commun ; il s'en inspire uniquement par la forme et l'esprit.

Pour aller plus loin. Les élèves peuvent traiter une **nouvelle version** de ce problème dotée d'un **autocorrecteur des réponses**, et suivre un **cours complet**, sur cpgeacademy.org.

Présentation générale

Une *chaîne de blocs* (*blockchain*) est une structure de données répartie dans laquelle des blocs sont ajoutés les uns à la suite des autres. En cas de latence du réseau ou d'attaque, plusieurs blocs peuvent être proposés comme successeurs d'un même bloc : la structure obtenue n'est plus une liste mais un **arbre de blocs**, dont la racine est le *bloc génésis*.

Le protocole **Casper** (Buterin et Griffith, 2019) est une surcouche de *finalisation* qui sélectionne, dans cet arbre, une unique chaîne canonique. Pour des raisons d'efficacité, Casper ne raisonne pas sur l'arbre complet mais sur le sous-arbre des **points de contrôle** (*checkpoints*) : le bloc génésis est un point de contrôle, et tout bloc dont le numéro dans l'arbre est un multiple exact de 100 en est un également.

Un ensemble de **validateurs** possédant chacun un *dépôt* (une somme immobilisée) émettent des **votes**. Lorsqu'une proportion suffisante du dépôt total soutient un même vote, un *lien de supermajorité* est créé ; de proche en proche, des points de contrôle deviennent *justifiés* puis *finalisés*. Deux règles, appelées **commandements de Casper**, interdisent certains votes ; tout validateur qui les enfreint perd la totalité de son dépôt (*slashing*).

Ce problème étudie, de manière algorithmique et mathématique, les objets et les propriétés de ce protocole. Il comporte **cinq parties largement indépendantes**, de difficulté croissante. Les fonctions écrites dans une question peuvent être librement réutilisées dans les questions suivantes.

Notations et conventions

- L'arbre des points de contrôle possède n nœuds numérotés de 0 à $n - 1$. Le nœud 0 est la **racine** r (le bloc génésis).
- L'arbre est représenté par une liste Python `parent` de longueur n telle que `parent[0] == -1` et, pour tout $c \geq 1$, `parent[c]` est le numéro du père de c . On suppose toujours :

$$\forall c \geq 1, \quad 0 \leq \text{parent}[c] < c.$$

- La **hauteur** $h(c)$ d'un point de contrôle c est le nombre d'arêtes du chemin de c à la racine. Ainsi $h(r) = 0$.

- Un point de contrôle a est un **ancêtre** de b si a appartient au chemin de b à la racine (donc b est ancêtre de lui-même). Deux points de contrôle a et b sont dits **en conflit** lorsque aucun des deux n'est ancêtre de l'autre.
- Les validateurs sont identifiés par des clés. Les dépôts sont donnés par un dictionnaire Python `depots` associant à chaque validateur un entier *strictement positif*. On note $T = \sum_{\nu} d_{\nu}$ le **dépôt total** et, pour un ensemble E de validateurs, $w(E) = \sum_{\nu \in E} d_{\nu}$.
- Un ensemble E de validateurs est une **supermajorité** lorsque $w(E) \geq \frac{2}{3} T$.
- Un **vote** est un triplet Python (v, s, t) où v est un validateur, s la *source* et t la *cible*. Le vote est **valide** si v appartient au jeu de validateurs et si s est un ancêtre *strict* de t . Sauf mention contraire, tous les votes manipulés sont valides ; en particulier $h(s) < h(t)$.
- En Python, seules les constructions du programme officiel sont autorisées : types de base, listes, tuples, dictionnaires, ensembles, boucles, fonctions, récursivité, ainsi que les modules `math` et `random`. Les fonctions `sorted` et `sort` sont autorisées et l'on admettra qu'elles trient une liste de m éléments en $O(m \log m)$.
- On admet que les opérations élémentaires sur un dictionnaire (lecture, écriture, test d'appartenance) s'effectuent en $O(1)$ en moyenne.
- Toutes les complexités demandées sont des complexités *en temps* dans le pire des cas, sauf indication explicite.

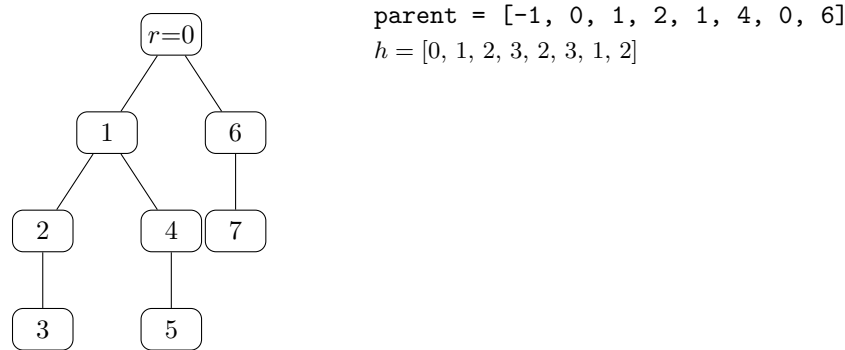


Figure 1 — L'arbre de points de contrôle A_0 utilisé comme exemple dans tout le sujet.

Le sujet comporte 5 parties et 54 questions. Il est vivement conseillé de traiter les parties dans l'ordre, mais elles restent largement indépendantes.

Partie I — Points de contrôle, dépôts et fuite d'inactivité

I.1 — Hauteurs

Un bloc de numéro n (entier naturel) est un point de contrôle si et seulement si n est un multiple de 100 ; sa hauteur de point de contrôle vaut alors $n/100$.

Question 1. Écrire deux fonctions `est_checkpoint(n)` et `hauteur_bloc(n)` qui prennent en argument un entier naturel n et renvoient respectivement le booléen indiquant si le bloc n est un point de contrôle, et l'entier $n/100$ (on supposera dans le second cas que n est bien un multiple de 100). Préciser leur complexité.

Question 2. Écrire une fonction `hauteur(parent, c)` qui calcule $h(c)$ en remontant les liens de parenté. Exhiber un **variant de boucle** entier et positif, et en déduire la terminaison de la fonction. Donner sa complexité en fonction de $h(c)$.

Question 3. On souhaite calculer d'un seul coup les hauteurs de *tous* les points de contrôle. Écrire une fonction `toutes_hauteurs(parent)` qui renvoie la liste $\mathbf{h}$ telle que $\mathbf{h}[c]$ vaut $h(c)$, en une **unique** boucle de $c = 0$ à $n - 1$. Justifier la correction de l'algorithme en énonçant un invariant de boucle, et préciser à quel endroit l'hypothèse `parent[c] < c` est indispensable. Donner la complexité.

Question 4. Vérifier sur l'arbre A_0 de la Figure 1 que `toutes_hauteurs` renvoie bien la liste h indiquée.

I.2 — Dépôts et supermajorités

Question 5. Écrire une fonction `poids(depots, E)` qui renvoie $w(E)$, puis une fonction `est_supermajorite(de E, T)` qui renvoie `True` si et seulement si E est une supermajorité. On exige que le test soit effectué **sans aucun calcul en virgule flottante** ; justifier l'équivalence utilisée.

Question 6. On considère les dépôts `depots = {a : 30, b : 30, c : 20, d : 20}`. Déterminer T , puis dire lesquels des ensembles $\{a, b\}$, $\{a, b, c\}$, $\{a, c, d\}$ sont des supermajorités.

Question 7. (Lemme d'intersection.) Soient E_1 et E_2 deux supermajorités. Démontrer que

$$w(E_1 \cap E_2) \geq \frac{T}{3}.$$

Indication : majorer $w(E_1 \cup E_2)$ par T et utiliser $w(E_1 \cup E_2) = w(E_1) + w(E_2) - w(E_1 \cap E_2)$. Ce lemme est la clé de toute la partie IV ; on veillera à rédiger la démonstration complètement.

I.3 — La fuite d'inactivité

Lorsque plus d'un tiers des validateurs cessent de voter, plus aucun point de contrôle ne peut être finalisé. Casper prévoit alors une **fuite d'inactivité** : à chaque époque, un validateur qui ne vote pas et dont le dépôt vaut D perd la fraction pD , où $p \in]0, 1[$ est un réel fixé. On note D_k le dépôt d'un validateur inactif après k époques, avec $D_0 = D > 0$.

Question 8. Exprimer D_k en fonction de D , p et k . En déduire la valeur de la somme $\sum_{k=0}^{+\infty} p D_k$ et interpréter ce résultat.

Question 9. Soit S un réel tel que $0 < S < D$. Démontrer que l'ensemble des entiers k tels que $D_k \leq S$ est non vide et que son plus petit élément vaut

$$k^* = \left\lceil \frac{\ln(S/D)}{\ln(1-p)} \right\rceil.$$

Question 10. Écrire une fonction `nb_epoques(D, p, S)` qui renvoie k^* sans utiliser de logarithme, par une simple boucle. Exhiber un variant prouvant la terminaison, puis montrer que la complexité est un $O(\log(D/S))$.

Question 11. Application numérique : calculer k^* pour $D = 1000$, $p = 0,05$ et $S = 500$.

Partie II — Arbre de points de contrôle et sérialisation des votes

II.1 — Ancêtres, conflits, plus proche ancêtre commun

Question 12. Écrire une fonction `est_ancetre(parent, a, b)` qui renvoie `True` si et seulement si a est un ancêtre de b . Exhiber un variant de boucle et donner la complexité en fonction de $h(b)$.

Question 13. En déduire une fonction `conflit(parent, a, b)` qui teste si a et b sont en conflit. Sur l'arbre A_0 de la Figure 1, déterminer si les paires $(2, 4)$, $(3, 5)$ et $(1, 3)$ sont en conflit.

Question 14. Écrire une fonction `ancetre_commun(parent, a, b, h)`, où h est la liste des hauteurs, qui renvoie le **plus proche ancêtre commun** de a et b , c'est-à-dire l'ancêtre commun de plus grande hauteur. L'algorithme devra procéder en deux temps : égaliser les hauteurs, puis remonter simultanément. Donner sa complexité.

Question 15. Démontrer la correction de `ancetre_commun` en énonçant l'invariant de la troisième boucle. On pourra admettre le résultat suivant, que l'on énoncera précisément : si x et y ont même hauteur et sont tous deux ancêtres d'un même nœud, alors $x = y$.

Question 16. Démontrer l'équivalence : a et b sont en conflit si et seulement si `ancetre_commun(a, b)  $\notin$   $\{a, b\}$` .

Question 17. Écrire une fonction `vote_valide(parent, depots, vote)` qui teste la validité d'un vote (v, s, t) au sens des notations.

II.2 — Sérialisation à longueur variable des hauteurs

Un vote $\langle v, s, t, h(s), h(t) \rangle$ est diffusé sur le réseau ; il faut donc le *sérialiser*, c'est-à-dire le transformer en une suite d'octets. Comme les hauteurs sont le plus souvent petites, on n'utilise pas un codage sur un nombre fixe d'octets, mais le codage à *longueur variable* suivant, dit codage en base 128.

Définition 1. Soit $n \in \mathbb{N}$ et soit $n = \sum_{i=0}^{\ell-1} a_i 128^i$ son écriture en base 128, avec $0 \leq a_i \leq 127$, $a_{\ell-1} \neq 0$ si $n \geq 1$, et $\ell = 1$, $a_0 = 0$ si $n = 0$. Le **codage** de n est la liste d'octets

$$\text{encode}(n) = [a_0 + 128, a_1 + 128, \dots, a_{\ell-2} + 128, a_{\ell-1}].$$

Autrement dit, chaque octet porte 7 bits de poids faible de n , et son bit de poids fort vaut 1 pour tous les octets sauf le dernier.

Question 18. Écrire une fonction `encode(n)` conforme à cette définition. Donner `encode(0)`, `encode(127)`, `encode(128)` et `encode(300)`.

Question 19. Montrer que le nombre de tours de la boucle de `encode` vaut 1 si $n = 0$ et $\lfloor \log_{128} n \rfloor + 1$ si $n \geq 1$. Exhiber un variant de boucle. En déduire que la longueur de `encode(n)` est un $O(\log n)$.

Question 20. Écrire la fonction réciproque `decode(octets)`, de complexité linéaire en la longueur de la liste.

Question 21. Démontrer, par récurrence sur ℓ (le nombre d'octets produits), que pour tout $n \in \mathbb{N}$ on a `decode(encode(n)) = n`. On distinguera soigneusement le cas $n < 128$ du cas général.

Question 22. Le sujet suppose des hauteurs pouvant aller jusqu'à 10^9 . Combien d'octets au maximum le codage d'une hauteur consomme-t-il ? Comparer à un codage sur 8 octets et commenter l'intérêt du codage à longueur variable pour un protocole où des milliers de votes sont diffusés à chaque époque.

Partie III — Justification et finalisation

On rappelle les définitions du protocole.

Définition 2 (Casper).

- Un couple ordonné (s, t) de points de contrôle est un **lien de supermajorité**, noté $s \rightarrow t$, lorsque l'ensemble des validateurs ayant publié un vote valide de source s et de cible t forme une supermajorité. Un lien peut « sauter » des points de contrôle : $h(t) > h(s) + 1$ est autorisé.
- Un point de contrôle c est **justifié** si $c = r$, ou s'il existe un point de contrôle justifié c' tel que $c' \rightarrow c$.
- Un point de contrôle c est **finalisé** si $c = r$, ou si c est justifié et s'il existe un lien $c \rightarrow c'$ où c' est un fils *direct* de c (donc $h(c') = h(c) + 1$).

Question 23. Écrire une fonction `liens_supermajorite(votes, depots)` qui prend une liste de votes valides (éventuellement avec des doublons, un même validateur pouvant émettre plusieurs fois le même vote) et renvoie l'ensemble des couples (s, t) formant un lien de supermajorité. On veillera à ne compter le dépôt d'un validateur qu'une seule fois par couple (s, t) . Donner la complexité en fonction du nombre m de votes.

Question 24. Écrire une fonction `enfants_de(parent)` qui renvoie la liste E telle que $E[c]$ soit la liste des fils de c . Complexité ?

Question 25. Écrire une fonction **récursive** `sous_arbre(E, c)` renvoyant la liste des descendants de c (lui compris). Démontrer que le nombre total d'appels engendrés par `sous_arbre(E, 0)` vaut exactement n . Montrer ensuite, sur l'exemple d'un arbre réduit à un chemin, que le coût des concaténations peut être **quadratique** en n ; proposer alors une variante `sous_arbre_acc` utilisant un accumulateur, de complexité $O(n)$.

Question 26. Écrire une fonction `justifies(parent, liens)` renvoyant l'ensemble des points de contrôle justifiés. On utilisera un parcours en profondeur itératif à partir de la racine dans le graphe orienté dont les arcs sont les liens de supermajorité. Donner la complexité en fonction de n et du nombre $|L|$ de liens.

Question 27. Démontrer la correction de `justifies` : montrer par double inclusion que l'ensemble renvoyé est exactement l'ensemble des points de contrôle justifiés. Pour l'inclusion réciproque, on raisonnera par récurrence sur la longueur d'une chaîne de justification menant de r à c .

Question 28. Écrire une fonction `finalises(parent, liens, J, h)` renvoyant l'ensemble des points de contrôle finalisés, où J est l'ensemble des justifiés. Donner la complexité.

Question 29. Démontrer que si c est finalisé et $c \neq r$, alors le fils direct c' intervenant dans la définition est lui-même justifié.

Question 30. On considère l'arbre A_0 de la Figure 1, les dépôts de la question 6, et la liste de votes où chacun des validateurs a, b, c émet les trois votes $(0, 1)$, $(1, 2)$ et $(2, 3)$. Déterminer l'ensemble des liens de supermajorité, l'ensemble des justifiés, puis l'ensemble des finalisés. Justifier chaque appartenance.

III.4 — La règle de choix de fourche

La règle de choix de fourche de Casper s'énonce : *suivre la chaîne contenant le point de contrôle justifié de plus grande hauteur*.

Question 31. Écrire une fonction `choix_fourche(J, h)` renvoyant le point de contrôle justifié de plus grande hauteur. Complexité ?

Question 32. On admet la propriété (iv) démontrée en partie IV : *tant que moins d'un tiers des validateurs (en dépôt) n'enfreint aucun commandement, il existe au plus un point de contrôle justifié de hauteur donnée*.

- (a) Démontrer que tout point de contrôle justifié c est relié à la racine par une chaîne $r = b_0 \rightarrow b_1 \rightarrow \dots \rightarrow b_k = c$ de liens de supermajorité dont *tous* les termes sont justifiés ; en déduire que chaque b_i est un ancêtre de b_{i+1} et que la suite $(h(b_i))_{0 \leq i \leq k}$ est strictement croissante.
- (b) En déduire que le maximum calculé par `choix_fourche` est atteint en un **unique** nœud, et que la règle de choix de fourche est donc bien définie.

Partie IV — Détection des infractions et sûreté responsable

Définition 3 (Les deux commandements de Casper). Un validateur ν ne doit publier deux votes distincts $\langle \nu, s_1, t_1 \rangle$ et $\langle \nu, s_2, t_2 \rangle$ vérifiant l’une des deux conditions suivantes :

- (I) $h(t_1) = h(t_2)$ (deux votes distincts pour une même hauteur de cible) ;
- (II) $h(s_1) < h(s_2) < h(t_2) < h(t_1)$ (un vote strictement emboîté dans un autre).

Tout validateur enfreignant l’un de ces commandements perd la totalité de son dépôt.

IV.1 — Détection algorithmique

Question 33. Écrire une fonction `infraction_I(votes, h)` qui renvoie un témoin d’infraction au commandement (I) — c’est-à-dire un triplet (ν, t_1, t_2) avec $t_1 \neq t_2$ et $h(t_1) = h(t_2)$ tous deux ciblés par ν — ou `None` s’il n’y en a pas. La complexité devra être en $O(m)$ en moyenne, où m est le nombre de votes. Justifier.

Question 34. Écrire une fonction `infraction_II_naif(votes, h)` détectant une infraction au commandement (II) par double boucle. Donner sa complexité.

Question 35. On veut faire mieux. Soit un validateur ν et la liste L de ses votes, triée par hauteur de source croissante. Démontrer l’équivalence suivante : ν *enfreint (II)* si et seulement s’il existe un vote (s_2, t_2) de L tel que

$$\max\{h(t_1) : (s_1, t_1) \in L, h(s_1) < h(s_2)\} > h(t_2).$$

On prendra garde au fait que la condition $h(s_2) < h(t_2)$ est automatiquement vérifiée pour un vote valide.

Question 36. En déduire une fonction `infraction_II(votes, h)` de complexité $O(m \log m)$, utilisant un tri puis un unique parcours maintenant le maximum courant des $h(t)$ déjà rencontrés. On prendra soin de traiter les votes de *même* hauteur de source par groupes, afin de respecter l’inégalité stricte $h(s_1) < h(s_2)$. Énoncer l’invariant de la boucle principale et justifier la complexité.

Question 37. Sur l’arbre A_0 , un validateur ν publie les deux votes $(0, 3)$ et $(1, 2)$. Quel commandement enfreint-il ? Même question pour les votes $(0, 2)$ et $(0, 4)$.

IV.2 — Sûreté responsable

On suppose désormais que **moins d’un tiers** des validateurs (en dépôt) enfreint un commandement.

Question 38. Démontrer les propriétés suivantes.

- (i) Si $s_1 \rightarrow t_1$ et $s_2 \rightarrow t_2$ sont deux liens de supermajorité distincts, alors $h(t_1) \neq h(t_2)$.
- (ii) Si $s_1 \rightarrow t_1$ et $s_2 \rightarrow t_2$ sont deux liens de supermajorité distincts, alors l’inégalité $h(s_1) < h(s_2) < h(t_2) < h(t_1)$ est impossible.

Indication : appliquer le lemme d’intersection de la question 7 aux deux ensembles de validateurs soutenant les deux liens.

Question 39. En déduire :

- (iii) pour toute hauteur n , il existe au plus un lien de supermajorité $s \rightarrow t$ avec $h(t) = n$;
- (iv) pour toute hauteur n , il existe au plus un point de contrôle justifié de hauteur n .

Question 40. (Théorème de sûreté responsable.) Soient a_m et b_n deux points de contrôle finalisés *en conflit*, de fils directs justifiés respectifs a_{m+1} et b_{n+1} . Quitte à échanger les rôles, on suppose $h(a_m) < h(b_n)$. En considérant la chaîne de justification $r \rightarrow b_{i_1} \rightarrow b_{i_2} \rightarrow \dots \rightarrow b_n$ et en introduisant le plus petit indice j tel que $h(b_j) > h(a_{m+1})$, démontrer que l'on aboutit à une contradiction avec (ii) ou (I). Conclure : *deux points de contrôle en conflit ne peuvent pas être tous deux finalisés sans qu'au moins un tiers du dépôt total ne soit détruit.*

Question 41. Pourquoi le cas $h(a_m) = h(b_n)$ se traite-t-il immédiatement ? Quel commandement est alors enfreint ?

IV.3 — Recherche probabiliste d'un validateur fautif

Un client léger ne peut pas vérifier tous les validateurs. Il tire donc au hasard, **avec remise et de façon indépendante**, des validateurs uniformément parmi les N validateurs, et vérifie pour chacun s'il est fautif. On suppose qu'une proportion $q \in]0, 1[$ des validateurs est fautive. On note X le rang du premier tirage amenant un validateur fautif.

Question 42. Justifier que X suit une loi géométrique de paramètre q . Donner $P(X = k)$ pour $k \geq 1$, puis $P(X > k)$, et enfin $\mathbb{E}[X]$.

Question 43. En déduire que l'algorithme de recherche s'arrête **presque sûrement**, c'est-à-dire que $P(X = +\infty) = 0$.

Question 44. Écrire une fonction `recherche_fautif(validateurs, est_fautif)` qui implémente cette recherche à l'aide de `random.randrange`, où `est_fautif` est une fonction booléenne. Que vaut l'espérance du nombre d'appels à `est_fautif` lorsque exactement un tiers des validateurs est fautif ? Pourquoi ne peut-on pas majorer le nombre de tours dans le pire des cas ?

Partie V — Coût minimal d'une attaque (programmation dynamique)

D'après la question 7, une attaque réussie contre la sûreté de Casper coûte à l'attaquant au moins $T/3$. Un attaquant rationnel cherche donc à **minimiser** le dépôt qu'il met en jeu tout en réunissant une supermajorité. Cela conduit au problème d'optimisation suivant.

Définition 4 (COÛT-ATTAQUE). *Donnée :* une liste $d = [d_0, \dots, d_{n-1}]$ d'entiers strictement positifs (les dépôts) et un entier $C \geq 0$ (le seuil, en pratique $C = \lceil 2T/3 \rceil$).

Question : déterminer $\mu(d, C) = \min \left\{ \sum_{i \in S} d_i : S \subseteq \{0, \dots, n-1\}, \sum_{i \in S} d_i \geq C \right\}$, avec $\mu = +\infty$ si aucun S ne convient.

Question 45. Justifier que $C = \lceil 2T/3 \rceil$ est bien le plus petit entier C tel que $w(E) \geq C \iff E$ est une supermajorité. Écrire l'expression Python de C à partir de `T` en n'utilisant que des divisions entières.

Question 46. Le problème de **décision** associé est : « existe-t-il S avec $\sum_{i \in S} d_i = C$? » (problème SOMME-DE-SOUS-ENSEMBLE). Démontrer que ce problème appartient à la classe **NP** en explicitant un certificat et un vérificateur polynomial. On rappelle (et on admettra) que SOMME-DE-SOUS-ENSEMBLE est NP-complet ; que peut-on en déduire quant à l'existence d'un algorithme polynomial pour COÛT-ATTAQUE, sous l'hypothèse $P \neq NP$?

Question 47. On pose, pour $0 \leq i \leq n$ et $0 \leq j \leq C$:

$$m_i(j) = \min \left\{ \sum_{k \in S} d_k : S \subseteq \{0, \dots, i-1\}, \min(C, \sum_{k \in S} d_k) = j \right\} \in \mathbb{N} \cup \{+\infty\}.$$

Autrement dit, $m_i(j)$ est la plus petite somme réalisable avec les i premiers dépôts dont la valeur *écrêtée* à C vaut exactement j . Donner m_0 , puis établir la relation de récurrence exprimant m_{i+1} en fonction de m_i . Expliquer pourquoi l'écrêtage $\min(C, \cdot)$ est nécessaire pour borner la taille du tableau, et pourquoi il ne fait perdre aucune solution optimale.

Question 48. Exprimer $\mu(d, C)$ à l'aide de m_n .

Question 49. Écrire une fonction `cout_min_attaque(d, C)` implémentant cette récurrence avec un **unique** tableau de taille $C + 1$ mis à jour en place. Justifier précisément le **sens de parcours** de l'indice j et montrer sur un exemple ce qui se produirait si l'on parcourait dans l'autre sens.

Question 50. Démontrer la correction de l'algorithme en énonçant et en prouvant, par récurrence sur le nombre i de dépôts déjà traités, l'**invariant** vérifié par le tableau à la fin de la i -ème itération de la boucle externe. Justifier également la terminaison.

Question 51. Donner la complexité en temps et en espace de `cout_min_attaque`. Expliquer pourquoi cet algorithme est dit **pseudo-polynomial** et pourquoi cela ne contredit pas la question 46 (NP-complétude). Que devient la complexité si les dépôts sont exprimés en *wei*, la plus petite unité de l'ether, soit 10^{18} wei par ether ?

Question 52. Application numérique : on prend $d = [12, 7, 20, 5, 16]$. Calculer T , puis C , puis $\mu(d, C)$ en explicitant un sous-ensemble optimal et en détaillant le contenu du tableau à la fin de chaque itération pour la version réduite $d = [3, 5, 7]$, $C = 8$.

Question 53. Comparer $\mu(d, C)$ au seuil C sur cet exemple. Un attaquant peut-il toujours mobiliser *exactement* C ? Proposer un jeu de dépôts pour lequel $\mu(d, C) > C$ strictement, et le justifier. Rapprocher enfin ce résultat de la question 7 : quelle part du dépôt mobilisé est effectivement *détruite* lors d'une violation de la sûreté ?

Question 54. Question de synthèse (à traiter en quelques lignes). Casper garantit la *sûreté responsable* mais pas la *vivacité* en toutes circonstances. À la lumière des parties III, IV et V, expliquer en quoi le mécanisme de fuite d'inactivité de la partie I restaure la vivacité, et pourquoi il fragilise en retour la garantie de sûreté démontrée à la question 49.